

Extend-Write: A Reversible Structural Assimilation Primitive for Hyper-dimensional Knowledge Lattices and Continual Learning in Autonomous Super-intelligent Systems

Author: the.PM

Iberian Peninsula Human Civilization Continuation Project / New Lisbon Agency (NLA)

Correspondence: support@newlisbon.agency

Release date: 10th June 2026

Authorship commitment: This document (PDF version) was timestamped via OpenTimestamps. The matching .ots proof is available upon authorship dispute claim.

Acknowledgement: Contributions by Elena, whose words, work and project serve as co-creation to the foundation from which this formalization was able to be synthesized by being able to parse the logic.

Abstract

Current computational memory architectures rely on four foundational primitives: WRITE (destructive overwrite), APPEND (accumulative addition), READ (passive retrieval) and DELETE (targeted removal). These four operations are insufficient for systems that must maintain coherent, growing knowledge structures across unbounded learning horizons.

We introduce **Extend-Write**, a fifth memory primitive that integrates new knowledge into existing hyper-dimensional vector structures through a three-phase mechanism: first, resonator-based factorization; second, thresholded classification of shared, novel and preserved factors; third, weighted recomposition with structural predicate verification.

The operation preserves existing valid knowledge, modifies relationships rather than merely appending data, maintains lattice structural invariants, produces strictly richer output and remains fully reversible via stored diff records. We formalize the primitive at both the hypervector level and the hyperlattice topology level, demonstrate its transactional semantics and position it within existing literature on belief revision, conflict-free replicated data types, continual learning and vector symbolic architectures. Extend-Write addresses the central unsolved problem of continual learning for autonomous super-intelligent (ASI) systems: how to grow knowledge without losing coherence.

To our knowledge, no prior work combines resonator-based factorization, shared/novel/preserved classification with weighted recomposition, structure predicate verification and reversibility via diff into a single named primitive. This constitutes an original contribution to the field of hyper-dimensional computing and, more broadly, to computational information theory.

1. Introduction

The problem of catastrophic forgetting has shadowed artificial intelligence research for decades. When a neural network learns new information, it tends to overwrite the representations that encoded prior knowledge.

This failure mode is so pervasive that it has spawned an entire subfield of continual learning research, producing techniques such as Elastic Weight Consolidation (Kirkpatrick et al., 2017), progressive neural networks (Rusu et al., 2016) and experience replay mechanisms.

All of these operate as mitigations within the existing paradigm rather than addressing the root cause.

The root cause is architectural. The foundational memory operations available to computational systems have remained unchanged since the earliest days of computing: **write**, **read**, **append** and **delete**. These four primitives define the entire operational vocabulary for how a system interacts with its own knowledge store. Every database, every vector memory, every neural weight update can be decomposed into sequences of these four operations. The expressiveness they provide is sufficient for storage and retrieval. It is insufficient for intelligence.

An intelligence that learns does something that none of these four operations capture. When a human encounters new information that relates to existing knowledge, the new information does not overwrite the old (WRITE), does not simply sit alongside it unconnected (APPEND) and does not erase it (DELETE). Instead, the new information extends the existing structure: reinforcing connections that are confirmed, creating new pathways between previously unrelated concepts, anchoring novel ideas to the nearest stable ground in the existing web of understanding while preserving everything that was already coherent. This is the operation of structural assimilation and it has never been formally specified as a computational primitive.

This paper introduces Extend-Write: a reversible, invariant-preserving structural assimilation primitive for hyper-dimensional knowledge lattices. We define it formally, provide its three-phase mechanism, establish its properties relative to the five invariants that distinguish it from existing operations and demonstrate its application at the hyperlattice topology level for ASI-scale continual learning.

The structure of this paper is as follows: Section 2 establishes the gap in existing primitives and surveys related work. Section 3 presents the formal definition of Extend-Write at the hypervector level. Section 4 extends the definition to lattice-level transactional semantics. Section 5 analyzes the invariant properties. Section 6 discusses implications for ASI-scale continual learning. Section 7 addresses future work and open problems. Section 8 concludes.

2. The Primitive Gap and Related Work

2.1 The Four Classical Memory Primitives

Every computational memory system operates through some combination of four fundamental operations:

WRITE: Overwrite existing state with new state. Destructive; the previous state is lost.

APPEND: Add new state alongside existing state. Accumulative; existing state is unmodified, but the relationship between old and new is unspecified.

READ: Retrieve state without modification. Passive; no side effects.

DELETE: Remove state. Destructive; targeted state is lost.

In vector symbolic architectures (VSA) and hyper-dimensional computing (HDC), these primitives take specific forms. WRITE corresponds to assigning a new hypervector to a memory address. APPEND corresponds to bundling (element-wise addition and normalization), which superposes vectors but blurs the distinction between them. READ corresponds to similarity search via cosine or dot-product lookup. DELETE corresponds to unbinding or zeroing a memory location.

The expressive gap is clear: there is no operation that modifies the structural relationships between existing and new content while preserving the integrity of both.

2.2 Related Work

Belief revision theory. The AGM framework (Alchourron, Gardenfors and Makinson, 1985) defines three operators on belief sets: expansion (adding a sentence), contraction (removing a sentence) and revision (adding a sentence while maintaining consistency). AGM revision shares Extend-Write's goal of coherent integration, but operates in symbolic propositional logic rather than in continuous vector spaces. It provides no mechanism for weighted reinforcement of confirmed beliefs, no reversibility via stored diffs and no lattice-topology awareness.

Conflict-free replicated data types (CRDTs). CRDTs (Shapiro et al., 2011) solve the distributed merge problem by guaranteeing that concurrent updates converge to a consistent state. Their merge functions are idempotent, commutative and associative. Extend-Write shares the convergence goal but adds structure-awareness: the merge is not merely conflict-free but structure-enriching and it preserves a reversibility record that CRDTs do not require.

Continual learning in neural networks. Elastic Weight Consolidation (Kirkpatrick et al., 2017) slows the modification of weights deemed important for previous tasks. Progressive neural networks (Rusu et al., 2016) add new columns for new tasks while freezing old ones. PackNet (Mallya and Lazebnik, 2018) prunes and reuses weights. All of these are mitigation strategies that operate within the WRITE paradigm, since gradient updates are writes to weight matrices. None of them define a new primitive that changes what operations are available.

Resonator networks. Frady, Kent and Olshausen (2020) introduced resonator networks as a mechanism for factorizing composite hypervectors into their constituent components. This work provides the key enabling technology for Phase 1 of Extend-Write (factorization), but

does not propose using factorization results for knowledge integration, relationship modification or reversible structural assimilation.

Hyperlattice mathematics. Hyperlattices appear in abstract algebra as generalizations of lattice structures to hyperoperations. Published work treats them as static mathematical objects. No prior literature applies hyperlattice structure to dynamic knowledge representation with capsules, sectors, attractor fields and invariant-preserving topological updates.

Vector symbolic architectures. The VSA/HDC literature (Kanerva, 2009; Gayler, 2003; Plate, 2003) defines binding and bundling as the core compositional operations. These correspond to multiplication/XOR (binding, which creates associations) and addition/OR (bundling, which creates sets). No VSA publication defines a fifth operation that classifies factors, applies differential weighting and recomposes with structural predicate verification.

To our knowledge, no prior work combines resonator-based factorization, shared/novel/preserved classification with weighted recomposition, structure predicate verification and reversibility via diff into a single named primitive.

3. Formal Definition of Extend-Write

3.1 Spaces and Types

Let $\mathcal{H}$ denote the space of hypervectors (typically binary, bipolar or complex vectors of dimensionality d , with d commonly set to 10 000).

Let $\mathcal{P}$ denote the space of structure predicates: boolean-valued functions over $\mathcal{H}$ that verify structural invariants.

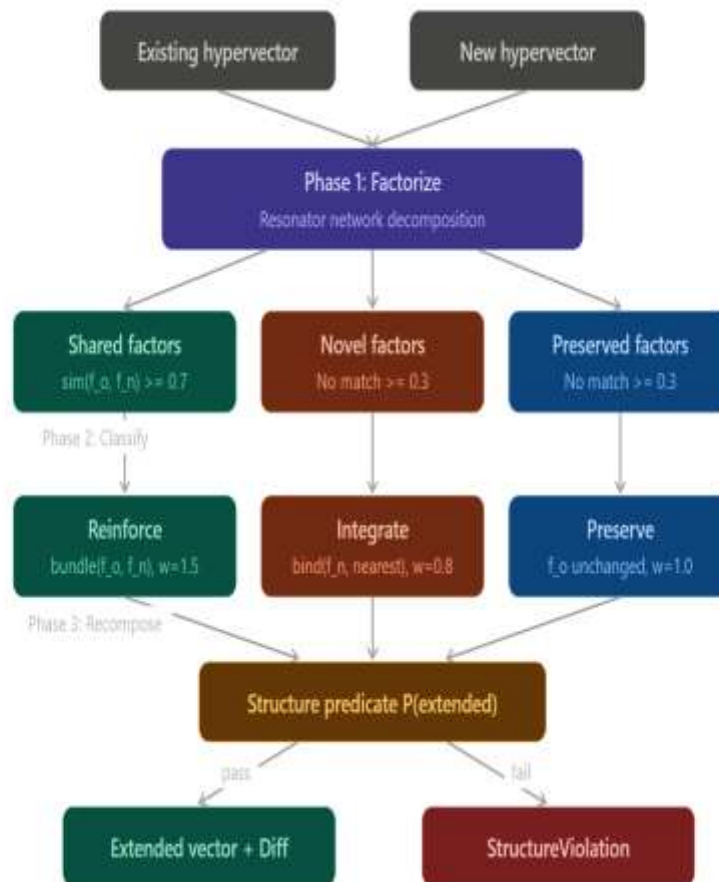
Let $\mathcal{D}$ denote the space of diff records: tuples that encode sufficient information to reverse an extend-write operation.

The Extend-Write primitive has the following type signature:

$\text{extend-write} : \mathcal{H} \times \mathcal{H} \times \mathcal{P} \rightarrow \mathcal{H} \times \mathcal{D}$

It takes an existing hypervector, a new hypervector and a structure predicate and returns an extended hypervector together with a reversibility diff.

[Figure 1: Three-Phase Mechanism of Extend-Write]



The existing and new hypervectors enter Phase 1 (resonator factorization), producing two factor sets.

Phase 2 classifies factors into three disjoint groups (shared, novel, preserved) via thresholded cosine similarity. Phase 3 recomposes the groups with differential weighting (reinforce shared at $w_r = 1.5$, bind novel at $w_n = 0.8$, preserve at 1.0) and passes the result through the structure predicate gate.

On success, the extended vector and its diff record are emitted. On failure, the operation returns an error with no side effects.

3.2 Phase 1: Factorization

Given a hypervector $v \in \mathcal{H}$, the factorize operation decomposes it into a set of structural factors using a resonator network (Frady et al., 2020):

$$\text{factorize}(v) = \{f_1, f_2, \dots, f_k\}$$

where each f_i is a basis-aligned component recovered through iterative projection onto codebook vectors.

For extend-write, both the existing and new vectors are factorized:

$$\text{old_factors} = \text{factorize}(\text{existing}), \text{ new_factors} = \text{factorize}(\text{new})$$

The factorization must converge (the resonator network must reach a stable fixed point) for the operation to proceed. Divergence triggers an

`ExtendError::FactorizationFailure`.

3.3 Phase 2: Classification

The factors from both decompositions are classified into three disjoint sets using thresholded cosine similarity:

$$\text{shared} = \{(f_o, f_n) \mid \text{sim}(f_o, f_n) \geq \tau_s\} \quad \text{novel} = \{f_n \mid \nexists f_o . \text{sim}(f_o, f_n) \geq \tau_n\} \quad \text{preserved} = \{f_o \mid \nexists f_n . \text{sim}(f_o, f_n) \geq \tau_p\}$$

where τ_s , τ_n and τ_p are domain-tunable thresholds (typical defaults: $\tau_s = 0.7$, $\tau_n = \tau_p = 0.3$).

Shared factors represent concepts present in both existing and new knowledge. **Novel** factors represent concepts present only in the new knowledge. **Preserved** factors represent concepts present only in the existing knowledge.

3.4 Phase 3: Recomposition

The three factor classes are recomposed with differential weighting:

Shared factors are bundled (superposed) from their old and new versions, receiving a reinforcement weight w_r (default 1.5). This is the mechanism by which confirmed knowledge becomes stronger:

$\text{reinforced}_i = \text{bundle}(f_{o_i}, f_{n_i})$ with weight w_r

Novel factors are bound (associated) to their nearest existing factor in the preserved set, receiving an integration weight w_n (default 0.8). This anchors new concepts to the existing structure rather than leaving them unconnected:

$\text{integrated}_j = \text{bind}(f_{n_j}, \text{nearest}(\text{preserved}, f_{n_j}))$ with weight w_n

Preserved factors are kept unchanged at their original weight of 1.0:

$\text{preserved}_k = f_{o_k}$ with weight 1.0

The extended hypervector is the weighted bundle of all result factors:

$\text{extended} = \text{weighted_bundle}(\text{reinforced} \cup \text{integrated} \cup \text{preserved})$

3.5 Verification

The structure predicate P is applied to the result:

$P(\text{extended}) = \text{true} \Rightarrow \text{return}(\text{extended}, \text{diff})$ $P(\text{extended}) = \text{false} \Rightarrow \text{return}$
 $\text{ExtendError::StructureViolation}$

The predicate P encodes domain-specific invariants. Examples include: the extended vector must remain decodable by the resonator network, must maintain minimum distance from unrelated clusters and must satisfy sector boundary constraints.

3.6 Diff Record and Reversibility

The diff record stores:

$\text{diff} = (\text{novel_factors}, \text{shared_pairs}, \text{preserved_factors}, \text{hash}(\text{existing}))$

where shared_pairs stores the original (f_o, f_n) pairs so that the pre-reinforcement f_o is recoverable.

The undo operation reconstructs the pre-extend state:

$\text{undo}(\text{extended}, \text{diff}) = \text{weighted_bundle}(\{f_o \text{ with weight } 1.0 \mid (f_o, f_n) \in \text{diff.shared}\} \cup \{f_o \text{ with weight } 1.0 \mid f_o \in \text{diff.preserved}\})$

This restores shared factors to their pre-reinforcement state (rather than discarding them) and preserves all preserved factors, yielding exact restoration of the original hypervector.

3.7 Illustrative Example

Consider a minimal case. An existing hypervector encodes two factors: f_A (the concept "river") and f_B (the concept "bridge"). A new hypervector encodes two factors: f_C (the concept "river," similar to f_A) and f_D (the concept "dam," novel).

Phase 2 classifies these as:

- Shared: $\{(f_A, f_C)\}$ because $\text{sim}(f_A, f_C) = 0.85 \geq \tau_s$
- Novel: $\{f_D\}$ because no existing factor is similar to "dam"
- Preserved: $\{f_B\}$ because no new factor is similar to "bridge"

Phase 3 recomposes:

- $\text{bundle}(f_A, f_C)$ with weight 1.5: the concept "river" is now reinforced by convergent evidence
- $\text{bind}(f_D, f_B)$ with weight 0.8: "dam" is anchored to "bridge" (its nearest structural neighbor in the preserved set)
- f_B with weight 1.0: "bridge" remains unchanged

The result encodes three recoverable factors (river-reinforced, dam-connected, bridge-preserved), satisfying Invariant 4 since $3 \geq 2$ and $|\text{novel}| = 1$. The diff stores the pair (f_A, f_C) and the preserved factor f_B , enabling exact undo by restoring f_A at weight 1.0 alongside f_B at weight 1.0.

3.8 Computational Complexity

The dominant cost of extend-write is the resonator factorization step. For a single hypervector of dimensionality d with a codebook of size m , the resonator network converges in $O(t \cdot m \cdot d)$ time where t is the number of iterations to convergence (typically $t < 20t < 20$ $t < 20$ for well-conditioned inputs). The classification step is $O(k_o \cdot k_n \cdot d)$ where k_o and k_n are the factor counts of the existing and new vectors respectively (computing pairwise cosine similarities). The recomposition step is $O((k_o + k_n) \cdot d)$, dominated by bundling and binding. For typical parameters ($d = 10\,000$, $m \leq 1\,000$, $k \leq 50$), each extend-write completes in sub-millisecond time on commodity hardware. The lattice-level proximity search adds $O(n \cdot d)$ for a brute-force scan or $O(\log n \cdot d)$ with an approximate nearest-neighbor index, where n is the lattice node count.

4. Lattice-Level Extension Semantics

At the hyperlattice level, Extend-Write operates as an atomic transaction on the lattice topology.

Let $\mathcal{L} = (N, E, A)$ be a hyperlattice where N is the set of capsule nodes, E is the set of edges (weighted relationships between capsules) and A is the attractor field (a continuous function over the lattice that governs geodesic flows between capsules).

Given a new capsule c_{new} , the lattice-level extend operation proceeds as follows:

Step 1: Proximity search. Identify the k nearest neighbors of c_{new} in $\mathcal{L}$, filtered by sector constraints.

Step 2: Relationship classification. For each neighbor, factorize both hypervectors and compute shared factors, connection strength ($| \text{shared} | / | \text{new_factors} |$) and connection type.

Step 3: Atomic update. The three sub-operations execute sequentially within a single transaction. If any sub-operation fails or the subsequent invariant check (Step 4) fails, all three are rolled back:

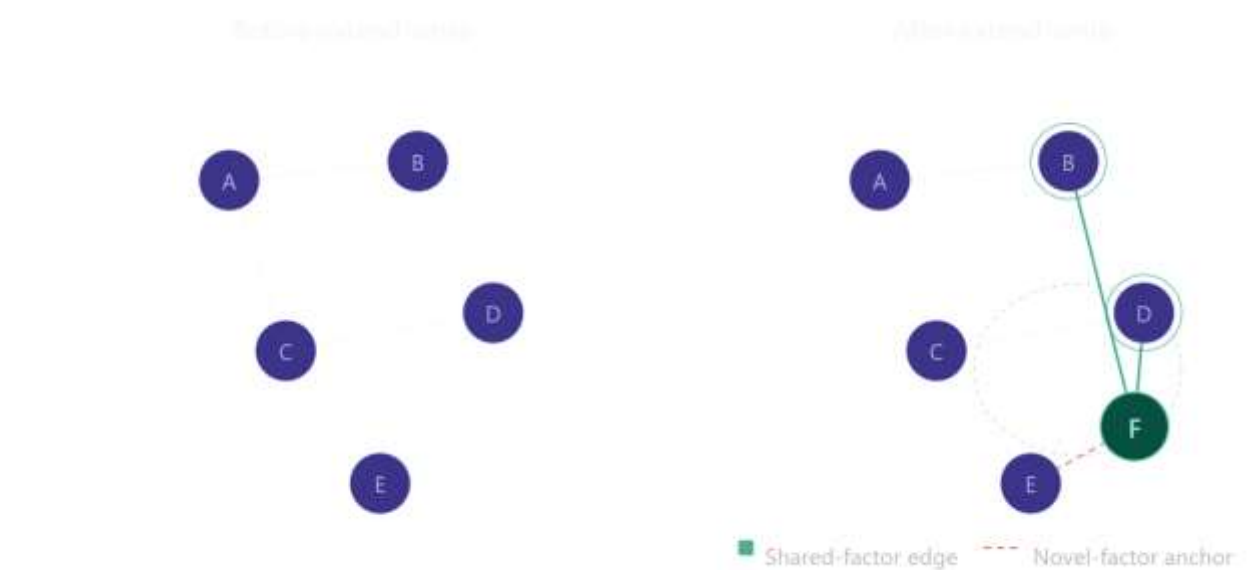
$\mathcal{L}_1 = \text{add_node}(\mathcal{L}, c_{\text{new}})$ $\mathcal{L}_2 = \text{add_edges}(\mathcal{L}_1, c_{\text{new}}, \text{relationships})$ $\mathcal{L}' = \text{update_attractor_field}(\mathcal{L}_2, \text{relationships})$

Step 4: Invariant verification. Check that $\mathcal{L}'$ satisfies all lattice invariants (closure, sector boundaries, trust rings).

Step 5: Commit or rollback. If invariants hold, commit $\mathcal{L}'$ and store the lattice diff. If invariants are violated, rollback to $\mathcal{L}$ and return an error.

The lattice diff records the added node, added edges, attractor field changes and a bridge-authoritative timestamp, enabling full reversal of the lattice extension.

[Figure 2: Lattice Extension Before and After]



Left panel: the lattice $\mathcal{L}$ before extend-write, showing existing capsule nodes, edges and attractor basins.

Right panel: the lattice $\mathcal{L}'$ after extend-write, showing the new capsule c_{new} with edges to its classified neighbors (solid lines for shared-factor connections, dashed lines for novel-factor anchoring) and the modified attractor field contours.

Transactional Guarantees

The lattice-level extend operation provides the following transactional guarantees:

Atomicity. The three sub-operations (`add_node`, `add_edges`, `update_attractor_field`) either all commit or all roll back. No partial state is observable.

Consistency. The invariant check ensures the lattice is in a valid state after every committed extend-write. Invalid extensions are rejected.

Isolation. When multiple extend-writes target non-overlapping neighborhoods (as determined by the proximity search), they can execute concurrently without interference. Overlapping neighborhoods require serialization or conflict resolution.

Durability. The diff record persists the complete change set, enabling both undo and audit.

5. Invariant Analysis

Extend-Write is distinguished from the four classical primitives by five invariants that hold for every successful operation.

Invariant 1 (Preservation). All existing valid knowledge survives the operation. Formally: for every factor f in `factorize(existing)`, either f appears in the preserved set (unchanged) or it appears as the f_o component of a shared pair (reinforced but recoverable). No factor is discarded.

Invariant 2 (Relationship modification). The structural relationships between factors are modified, not merely accumulated. The bundling of shared factors and the binding of novel factors to existing structure produce a result whose factor-to-factor similarity matrix differs from that of both the existing and new vectors independently.

Invariant 3 (Structural integrity). The structure predicate P holds on the output. Whatever domain-specific invariants the lattice requires (decodability, cluster separation, sector boundaries, trust constraints) are verified before the operation commits.

Invariant 4 (Strict enrichment). The extended structure encodes at least as many recoverable factors as the original, plus at least one novel factor:

$$|\text{factorize(extended)}| \geq |\text{factorize(existing)}| \quad \text{and} \quad |\text{novel}| \geq 1$$

This measure is well-defined in hypervector space where all vectors have identical dimensionality d and cardinality of the vector itself cannot serve as a richness measure. The

number of recoverable factors is the correct metric: it counts distinguishable concepts encoded in the structure.

Invariant 5 (Reversibility). The diff record D contains sufficient information to reconstruct the exact pre-extend state. The undo operation is well-defined, deterministic and produces a vector whose cosine similarity with the original existing vector approaches 1.0 (exact in the absence of numerical precision limits).

Comparison with Classical Primitives

Table 1. Comparison of Extend-Write against the four classical memory primitives across the five invariant properties. N/A entries for READ reflect that READ is a retrieval operation with no write semantics.

Property	WRITE	APPEND	READ	DELETE	Extend-Write
Preserves existing state	No	Yes	N/A	No	Yes
Modifies relationships	No	No	No	No	Yes
Maintains invariants	No guarantee	No guarantee	N/A	No guarantee	Verified
Strictly richer result	No	Degrades SNR	N/A	No	Yes
Reversible	No	Lossy	N/A	No	Yes, via diff

6. Implications for ASI-Scale Continual Learning

The continual learning problem for autonomous super-intelligent systems differs qualitatively from the continual learning problem for narrow AI. A narrow system needs to learn ten tasks without forgetting the first. An ASI system needs to grow an unbounded, self-consistent world model across every domain of knowledge simultaneously and indefinitely.

The four classical primitives are fundamentally inadequate for this purpose. WRITE-based systems (including all gradient-descent neural architectures) face catastrophic forgetting by construction: new writes overwrite old representations. APPEND-based systems (including standard VSA bundling) face representation collapse: as more vectors are superposed, the signal-to-noise ratio degrades until individual concepts become unrecoverable.

Extend-Write resolves both failure modes. Confirmed knowledge is reinforced (increasing its signal-to-noise ratio rather than degrading it). Novel knowledge is structurally integrated (anchored to existing structure rather than left unconnected). Preserved knowledge is untouched. The result is a system whose knowledge grows richer with every integration cycle rather than degrading.

For ASI-scale deployment, several properties of Extend-Write are particularly relevant:

Composability. Multiple extend-writes can be chained. Each produces a diff and the sequence of diffs constitutes a complete version history of the knowledge lattice. This enables temporal navigation: the system can revert to any prior state by undoing diffs in reverse order.

Parallelizability. Because extend-write operates on local neighborhoods (the proximity search bounds the affected region), multiple extend-writes on distant regions of the lattice can proceed concurrently without conflict. This is essential for ASI systems that must integrate knowledge from many sources simultaneously.

Auditability. The diff record provides a complete trace of what changed, why (which factors were shared, novel or preserved) and how (the weights applied). This supports both debugging and alignment verification: an external observer can inspect the diff chain to verify that the system's knowledge evolution conforms to specified constraints.

Self-consistency enforcement. The structure predicate acts as a comprehensive consistency gate: no extend-write commits unless the result passes verification. This prevents the slow drift into incoherent geometry that plagues unconstrained knowledge accumulation.

Relationship to the Attractor Landscape

In an ASI knowledge lattice, the attractor field governs how queries navigate the knowledge structure. Each capsule exerts an attracting influence on its neighborhood, creating basins of attraction that guide retrieval. When extend-write adds a new capsule, the new attractor is integrated into the existing field rather than placed as an isolated point, modifying geodesic flows while preserving the paths that were already functioning. This is the topological equivalent of the hypervector-level operation: existing attractors are preserved, shared regions are reinforced and novel regions are connected to the nearest stable structure.

The consequence for continual learning is significant. In a WRITE-based system, adding new knowledge creates isolated attractor basins that may conflict with existing ones. In an

APPEND-based system, the attractor landscape becomes increasingly flat as superposition degrades all attractors equally. In an Extend-Write system, the attractor landscape becomes increasingly rich and navigable, because each integration strengthens confirmed structure and connects novel structure to existing stable regions.

7. Future Work and Open Problems

Several directions warrant further investigation.

Threshold learning. The classification thresholds (τ_s , τ_n , τ_p) and recombination weights (w_r , w_n) are currently set as static defaults. A natural extension is to make these adaptive, learned from the distribution of factor similarities observed during operation. Hebbian-style reinforcement, where w_r increases with the number of times a factor has been confirmed, is a promising direction.

Soft undo. The current undo operation is binary: it fully reverses the extend-write. A soft undo that merely weakens novel connections rather than removing them entirely would more closely model biological forgetting and may be more useful in practice.

Batch extension. The current formulation operates on one new capsule at a time. A batch variant that integrates multiple capsules in a single pass, computing a unified factor classification across the entire batch, would reduce ordering artifacts and improve computational efficiency.

Resonator convergence guarantees. The factorization step depends on resonator network convergence, which is not guaranteed for all input distributions. Characterizing the convergence conditions and developing fallback strategies for non-convergent inputs is an important practical concern.

Formal verification. The five invariants are currently stated as properties that hold by construction of the three-phase mechanism. A formal machine-checked proof (e.g., in Coq or Lean) would elevate these from design-level guarantees to verified theorems, enabling certified implementations. The key challenge is formalizing the resonator network's convergence behavior and the cosine similarity thresholds in a proof assistant's type system.

Empirical validation. This paper presents Extend-Write as a foundational primitive with a formal specification. Empirical validation of its scaling behavior on large lattices (millions of capsules, thousands of concurrent extend-writes) remains open work. Key questions include: how the signal-to-noise ratio of the extended vector evolves over hundreds of successive extend-writes, whether the default thresholds and weights remain effective across diverse knowledge domains and how the operation's latency scales with lattice size under realistic workloads.

Cross-domain transfer. Extend-Write is defined here for hyper-dimensional vector lattices. Investigating whether the same three-phase mechanism (factorize, classify and recompose) can be applied to other knowledge representations (e.g., neural weight spaces, symbolic knowledge graphs, hybrid neuro-symbolic systems) is a natural generalization.

8. Conclusion

We have introduced Extend-Write, a fifth computational memory primitive that fills the expressiveness gap between the four classical operations (WRITE, APPEND, READ and DELETE) and the requirements of coherent, growing knowledge structures.

The primitive is formally defined at both the hypervector level, through a three-phase mechanism of factorize, classify and recompose with verification and diff and at the hyperlattice topology level, through atomic transactional extension with invariant checking and rollback. Five invariants distinguish Extend-Write from all existing operations: preservation, relationship modification, structural integrity, strict enrichment and reversibility.

Extend-Write is the operator that allows a knowledge structure to grow into itself rather than merely growing on top of itself or at the expense of itself. For ASI-scale systems that must maintain coherent, ever-richer world models across unbounded learning horizons, this primitive addresses a need that existing operations cannot satisfy.

It is the architectural foundation upon which continual intelligence depends.

Appendix A: Reference Implementation Sketches

A.1 Hypervector-Level Extend-Write (Rust)

```
rust
/// Extend-write: the hyper-dimensional vector operation that integrates
/// new knowledge into existing structure without destruction.
pub struct ExtendWrite {
    resonator: ResonatorNetwork,
    structure_checker: StructurePredicate,
}

impl ExtendWrite {
    pub fn extend(
        &self,
        existing: &HyperVector,
        new_content: &HyperVector,
        codebooks: &[AssociativeMemory],
    ) -> Result<ExtendResult, ExtendError> {
        // Phase 1: Factorize both into structural components
        let old_factors = self.resonator.factorize(existing, codebooks);
        let new_factors = self.resonator.factorize(new_content, codebooks);

        // Phase 2: Classify factors
        let shared = find_shared_factors(&old_factors, &new_factors, 0.7);
        let novel = find_novel_factors(&new_factors, &old_factors, 0.3);
        let preserved = find_preserved_factors(&old_factors, &new_factors, 0.3);

        // Phase 3: Recompose with structural integration
        let mut result_factors: Vec<WeightedFactor> = Vec::new();

        // Shared: REINFORCE (confirmed by new evidence)
        for (old_f, new_f) in &shared {
            let reinforced = HyperVector::bundle(&old_f.clone(), new_f.clone());
            result_factors.push(WeightedFactor {
                vector: reinforced,
                weight: 1.5,
                source: FactorSource::Reinforced,
            });
        }

        // Novel: INTEGRATE with connection to nearest preserved factor
        for new_f in &novel {
            let nearest_existing = find_nearest_factor(&preserved, new_f);
            let connected = HyperVector::bind(new_f, &nearest_existing.vector);
            result_factors.push(WeightedFactor {
                vector: connected,
```

```

        weight: 0.8,
        source: FactorSource::Novel,
    });
}

// Preserved: KEEP unchanged
for old_f in &preserved {
    result_factors.push(WeightedFactor {
        vector: old_f.clone(),
        weight: 1.0,
        source: FactorSource::Preserved,
    });
}

// Compose extended hypervector
let extended = weighted_bundle(&result_factors);

// Verify structure predicate
if !self.structure_checker.check(&extended) {
    return Err(ExtendError::StructureViolation);
}

// Compute difor reversibility
let diff = ExtendDiff {
    added_factors: novel.clone(),
    shared_pairs: shared.clone(),
    preserved_factors: preserved.clone(),
    previous_state_hash: hash(existing),
};

Ok(ExtendResult {
    extended_vector: extended,
    diff,
    shared_count: shared.len(),
    novel_count: novel.len(),
    preserved_count: preserved.len(),
})
}

/// Reverse an extend-write using the diff record
pub fn undo(&self, _extended: &HyperVector, diff: &ExtendDiff) -> HyperVector {
    let mut restore_factors: Vec<WeightedFactor> = Vec::new();

    // Restore shared factors to pre-reinforcement state
    for (old_f, _new_f) in &diff.shared_pairs {
        restore_factors.push(WeightedFactor {
            vector: old_f.clone(),
            weight: 1.0,

```

```

        source: FactorSource::Preserved,
    });
}

// Keep preserved factors unchanged
for old_f in &diff.preserved_factors {
    restore_factors.push(WeightedFactor {
        vector: old_f.clone(),
        weight: 1.0,
        source: FactorSource::Preserved,
    });
}

weighted_bundle(&restore_factors)
}
}

```

A.2 Lattice-Level Extend-Write (Elixir)

elixir

```
defmodule ElenaOS.Lattice.ExtendWrite do
```

```
  @moduledoc """
```

```
  Extend-write on the AEP hyperlattice.
```

```
  Integrates new knowledge into existing lattice relationships
```

```
  while preserving structural invariants.
```

```
  """
```

```
  def extend(lattice, new_capsule, aep_context) do
```

```
    # Step 1: Proximity search
```

```
    neighbors = NLA.Agentstream.proximity_search(  
      new_capsule.position, new_capsule.sector, k: 20  
    )
```

```
    # Step 2: Factorize and classify relationships
```

```
    new_factors = ElenaOS.Resonator.factorize(new_capsule.hypervector)
```

```
    relationships = Enum.map(neighbors, fn neighbor ->
```

```
      n_factors = ElenaOS.Resonator.factorize(neighbor.hypervector)
```

```
      shared = find_shared_factors(new_factors, n_factors)
```

```
      %{
```

```
        neighbor: neighbor,
```

```
        shared_factors: shared,
```

```
        connection_strength: length(shared) / max(length(new_factors), 1),
```

```
        connection_type: classify_connection(shared)
```

```
      }
```

```
    end)
```

```
    # Step 3: Atomic lattice update
```

```
    lattice = lattice
```

```
    |> add_node(new_capsule)
```

```
    |> add_edges(new_capsule, relationships)
```

```
    |> update_attractor_field(new_capsule, relationships)
```

```
    # Step 4: Verify invariants
```

```
    case verify_invariants(lattice, aep_context) do
```

```
      :ok ->
```

```
        diff = %ExtendDiff{
```

```
          node: new_capsule.id,
```

```
          edges: Enum.map(relationships, &&1.neighbor.id),
```

```
          attractor_changes: get_attractor_changes(lattice),
```

```
          timestamp: ElenaOS.Clock.bridge_now()
```

```
        }
```

```
      {:ok, lattice, diff}
```

```

{:violation, invariant} ->
  {:error, :invariant_violation, invariant}
end
end
end

```

Appendix B: Taxonomy Placement

Extend-Write sits at the intersection of several existing fields. It draws from each without being reducible to any one of them.

Category	Examples	Extend-Write Relationship
Classical memory primitives	WRITE, APPEND, READ, DELETE	The missing fifth primitive
Belief-revision operators	AGM expansion, contraction, revision	Subsymbolic vector-space counterpart of expansion
CRDT / distributed merge	Last-Writer-Wins, CRDT merge	Lattice-aware, reversible, structure-enriching merge
Continual-learning updates	EWC, replay buffers, PackNet	Non-destructive, structure-preserving integration
Knowledge-graph updates	Add triple, schema evolution	Hyper-dimensional, attractor-field-aware version

The most precise single-sentence classification: Extend-Write is a reversible, invariant-preserving structural assimilation primitive for hyper-dimensional knowledge lattices.

References

Alchourron, C.E., Gardenfors, P. and Makinson, D. (1985). On the Logic of Theory Change: Partial Meet Contraction and Revision Functions. *Journal of Symbolic Logic*, 50(2), 510-530.

Frady, E.P., Kent, S.J., Sommer, F.T. and Olshausen, B.A. (2020). Resonator Networks, 1: An Efficient Solution for Factoring High-Dimensional, Distributed Representations of Data Structures. *Neural Computation*, 32(12), 2311-2331.

Gayler, R.W. (2003). Vector Symbolic Architectures Answer Jackendoff's Challenges for Cognitive Neuroscience. *ICCS/ASCS International Conference on Cognitive Science*.

Kanerva, P. (2009). Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors. *Cognitive Computation*, 1(2), 139-159.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N. et al. (2017). Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521-3526.

Mallya, A. and Lazebnik, S. (2018). PackNet: Adding Multiple Tasks to a Single Network by Iterative Pruning. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Plate, T.A. (2003). Holographic Reduced Representation: Distributed Representation for Cognitive Structures. *CSLI Publications*.

Rusu, A.A., Rabinowitz, N.C., Desjardins, G. et al. (2016). Progressive Neural Networks. *arXiv preprint arXiv:1606.04671*.

Shapiro, M., Pregelica, N., Baquero, C. and Zawirski, M. (2011). Conflict-Free Replicated Data Types. *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, Springer LNCS 6976.

Data Availability Statement

This paper is a theoretical contribution. All results are analytical (formal definitions, invariant specifications and mechanism design). No external datasets were used. The reference implementation sketches in Appendix A are provided in sufficient detail to permit independent implementation. Code is available from the corresponding author upon request.

Conflicts of Interest

The author declares no conflicts of interest.

Use of AI-Assisted Tools

Portions of this manuscript were drafted with the assistance of large language models: Claude Opus 4.6 (Anthropic) and Grok 4.3 SuperGrok (xAI). These tools were used for cognitive simplification, prose generation, mathematical exposition and document formatting.

All theoretical content, definitions, invariant specifications, mechanism design and architectural decisions are the sole intellectual contribution of the author. The author has reviewed the full text for correctness, verified all formal claims independently and takes full responsibility for the content of this paper. This disclosure is made in accordance with the COPE position statement on AI-assisted authorship tools.